Staircase Traversal

AlgoExpert · Medium · Recursion

The problem

- Ways to climb `height` steps taking 1..maxSteps at a time.
- Sliding-window sum DP.

Intuition

- Ways to reach step n is the sum of ways for the last maxSteps steps, a sliding window.

Brute force vs optimal

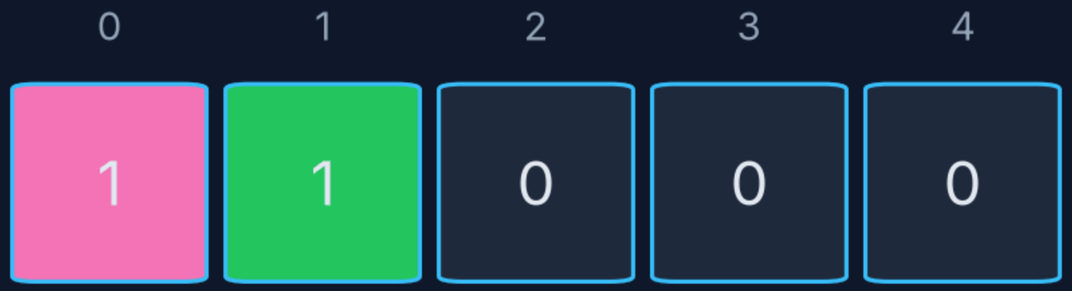
- Recursion over step sizes → $O(k^n)$
- Memo/DP summing last k → $O(n \cdot k)$
- Sliding window sum → $O(n)$

Approach

- 1. ways of zero is one
- 2. Keep a running window sum
- 3. Add the new value and drop the oldest

Worked example

ways to reach each step

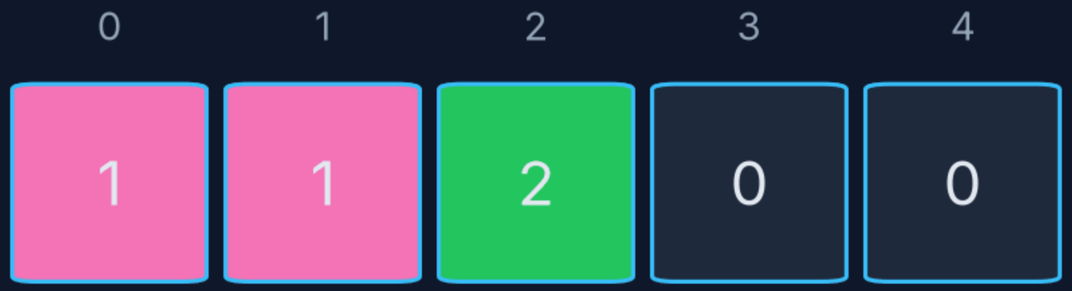



```
ways[1] = sum of the last 2 = 1
```

ways[1] = 1

Worked example

ways to reach each step

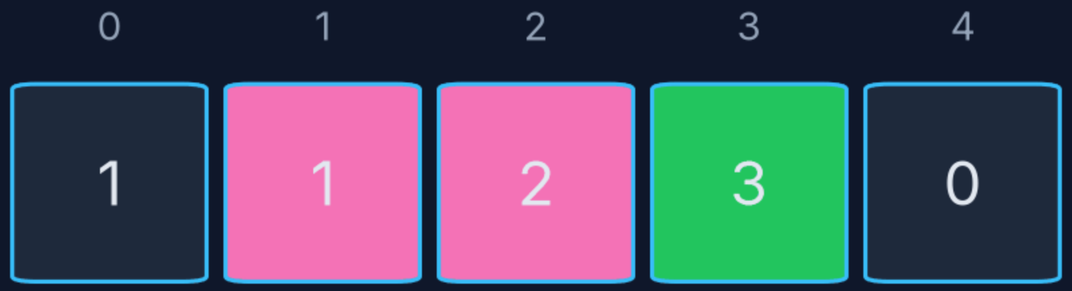



```
ways[2] = sum of the last 2 = 2
```

ways[2] = 2

Worked example

ways to reach each step

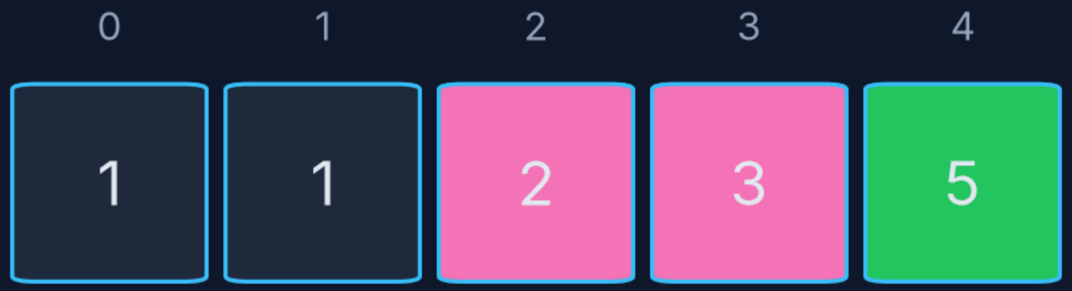



```
ways[3] = sum of the last 2 = 3
```

ways[3] = 3

Worked example

ways to reach each step




```
ways[4] = sum of the last 2 = 5
```

ways[4] = 5

Java solution

```java
public class StaircaseTraversal {
    public static int staircaseTraversal(int height, int maxSteps) {
        int[] ways = new int[height + 1];
        ways[0] = 1;
        int window = 0;
        for (int h = 1; h <= height; h++) {
            window += ways[h - 1];
            if (h - maxSteps - 1 >= 0) window -= ways[h - maxSteps - 1];
            ways[h] = window;
        }
        return ways[height];
    }
}
```

Complexity

- Time: O(n)
- Space: O(n)

Edge cases & pitfalls

- Ways[0] = 1
- Subtract the element leaving the window
- MaxSteps > height

Interview tips & follow-ups

- Notice Fibonacci for k = 2
- Follow-up: forbidden steps

Recap

- Ways to reach step n is the sum of ways for the last maxSteps steps, a sliding window.
- Complexity: O(n)